

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Линейные и циклические списки

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Целью работы является изучение структур данных «линейный список» и «циклический список», а также получение практических навыков их реализации.

2 Вариант задания

Дана последовательность неповторяющихся чисел $a_1, a_2, \dots, a_n$ и некоторое число c , принадлежащее данной последовательности. Составить 2 последовательности:

- Первая – все числа, находящиеся до указанного числа в обратном порядке.
- Вторая – все числа после указанного числа в прямом порядке

Вид списка – линейный двусвязный

3 Описание работы программы

Программа реализует структуру данных линейный двусвязный список и выполняет обработку последовательности чисел согласно варианту 11.

3.1 Структура данных

Каждый узел списка содержит три поля: числовое значение `data`, указатель на следующий элемент `next` и указатель на предыдущий элемент `prev`. Наличие двух указателей позволяет выполнять обход списка как в прямом, так и в обратном направлении.

3.2 Порядок работы программы:

Пользователь вводит количество элементов n и сами числа. Список заполняется функцией `addToEnd`, которая последовательно добавляет элементы в конец, связывая указатели `next` и `prev`.

Выводится исходный список.

Пользователю предлагается добавить или удалить элемент. Удаление реализовано в функции `removeByValue` — найденный узел исключается из цепочки путём перелинковки соседних узлов.

Пользователь вводит число `c`. Программа находит соответствующий узел в списке.

Формируются два результирующих списка: первый — обход от узла `c` назад по указателям `prev` (числа до `c` в обратном порядке), второй — обход от узла `c` вперёд по указателям `next` (числа после `c` в прямом порядке).

Оба результирующих списка выводятся на экран. В конце работы выделенная память освобождается.

4 Листинг программы, реализующей поставленную задачу с использованием заданных структур данных

```
#include <iostream>
using namespace std;

// Структура узла двусвязного списка
struct Node {
    int data;    // значение элемента
    Node* prev; // указатель на предыдущий элемент
    Node* next; // указатель на следующий элемент
};

// Добавить элемент в конец списка
void addToEnd(Node*& head, Node*& tail, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;
    newNode->prev = tail;

    // Если список не пустой — связываем с предыдущим хвостом
    if (tail != nullptr)
        tail->next = newNode;
    else
        head = newNode; // список был пуст

    tail = newNode; // новый элемент становится хвостом
}
```

```

// Удалить элемент по значению
bool removeByValue(Node*& head, Node*& tail, int value) {
    Node* cur = head;

    // Поиск элемента
    while (cur != nullptr) {
        if (cur->data == value) {

            // Перенастройка указателей соседних элементов
            if (cur->prev)
                cur->prev->next = cur->next;
            else
                head = cur->next;

            if (cur->next)
                cur->next->prev = cur->prev;
            else
                tail = cur->prev;

            delete cur; // освобождаем память
            return true;
        }

        cur = cur->next;
    }

    return false; // элемент не найден
}

// Вывести список
void printList(Node* head) {

    if (head == nullptr) {
        cout << "List is empty" << endl;
        return;
    }

    Node* cur = head;

    while (cur != nullptr) {
        cout << cur->data;

        if (cur->next)
            cout << " <-> ";

        cur = cur->next;
    }
}

```

```

        cout << endl;
    }

    // Освободить память списка
    void freeList(Node*& head, Node*& tail) {

        while (head != nullptr) {
            Node* tmp = head;
            head = head->next;
            delete tmp;
        }

        tail = nullptr;
    }

    int main() {

        Node* head = nullptr;
        Node* tail = nullptr;

        int n;

        // Ввод количества элементов
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter " << n << " unique numbers: ";

        for (int i = 0; i < n; i++) {
            int x;
            cin >> x;
            addToEnd(head, tail, x);
        }

        cout << "\nOriginal list: ";
        printList(head);

        // Добавление элемента
        cout << "\nAdd a new element? (1 - yes, 0 - no): ";
        int choice;
        cin >> choice;

        if (choice == 1) {
            int val;
            cout << "Enter value: ";
            cin >> val;

```

```

        addToEnd(head, tail, val);

        cout << "List after adding: ";
        printList(head);
    }

    // Удаление элемента
    cout << "\nRemove an element? (1 - yes, 0 - no): ";
    cin >> choice;

    if (choice == 1) {

        int val;

        cout << "Enter value to remove: ";
        cin >> val;

        if (removeByValue(head, tail, val))
            cout << "Element removed." << endl;
        else
            cout << "Element not found." << endl;

        cout << "List after removal: ";
        printList(head);
    }

    // Основное задание
    int c;

    cout << "\nEnter number c (from the sequence): ";
    cin >> c;

    // Поиск элемента c в списке
    Node* cNode = head;

    while (cNode != nullptr && cNode->data != c)
        cNode = cNode->next;

    if (cNode == nullptr) {
        cout << "Number " << c << " not found in the list!" << endl;

        freeList(head, tail);
        return 1;
    }

    // --- Первая последовательность: числа ДО c в обратном порядке ---
    Node* resHead1 = nullptr;
    Node* resTail1 = nullptr;

```

```

// Движение назад по двусвязному списку
Node* cur = cNode->prev;

while (cur != nullptr) {
    addToEnd(resHead1, resTail1, cur->data);
    cur = cur->prev;
}

// --- Вторая последовательность: числа ПОСЛЕ c в прямом порядке ---
Node* resHead2 = nullptr;
Node* resTail2 = nullptr;

cur = cNode->next;

while (cur != nullptr) {
    addToEnd(resHead2, resTail2, cur->data);
    cur = cur->next;
}

// Вывод результатов
cout << "\n=== Results ===" << endl;

cout << "Numbers before " << c << " in reverse order: ";
printList(resHead1);

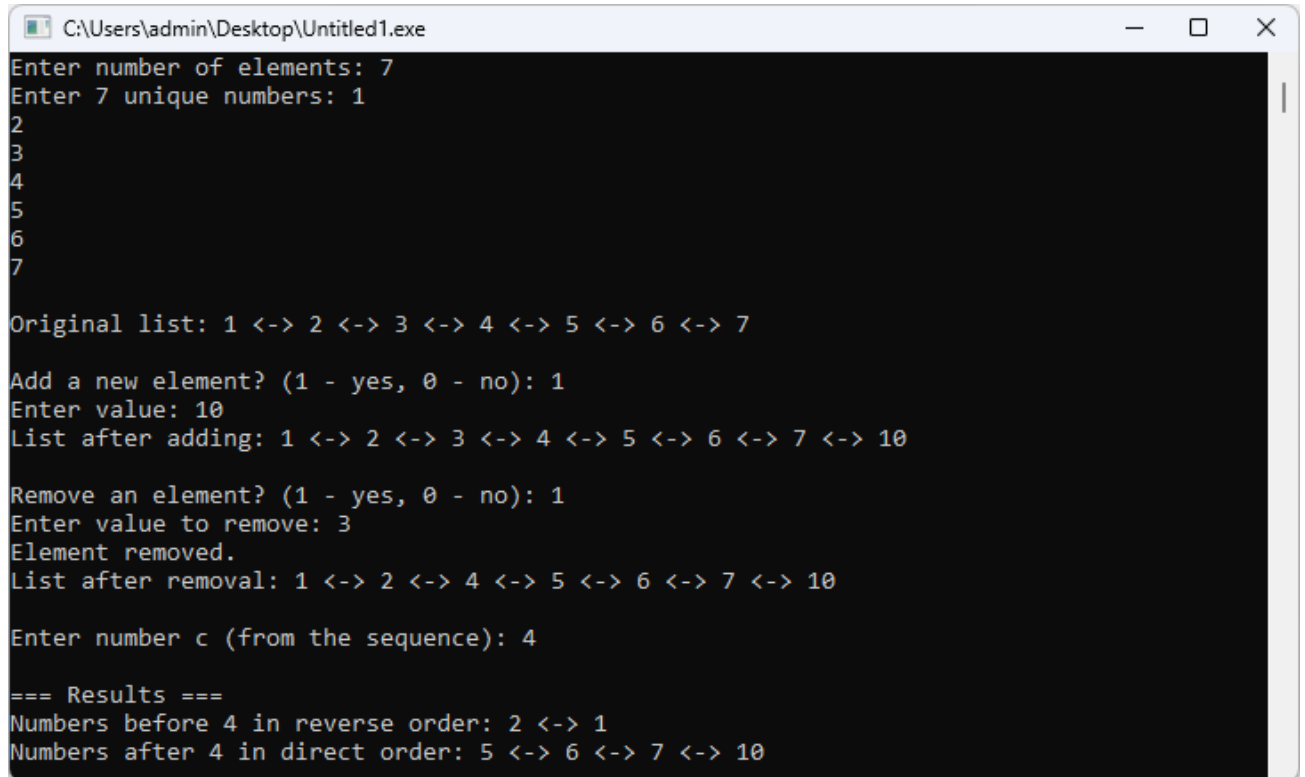
cout << "Numbers after " << c << " in direct order: ";
printList(resHead2);

// Освобождение памяти
freeList(head, tail);
freeList(resHead1, resTail1);
freeList(resHead2, resTail2);

return 0;
}

```

5 Контрольные примеры



```
C:\Users\admin\Desktop\Untitled1.exe
Enter number of elements: 7
Enter 7 unique numbers: 1
2
3
4
5
6
7

Original list: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> 7

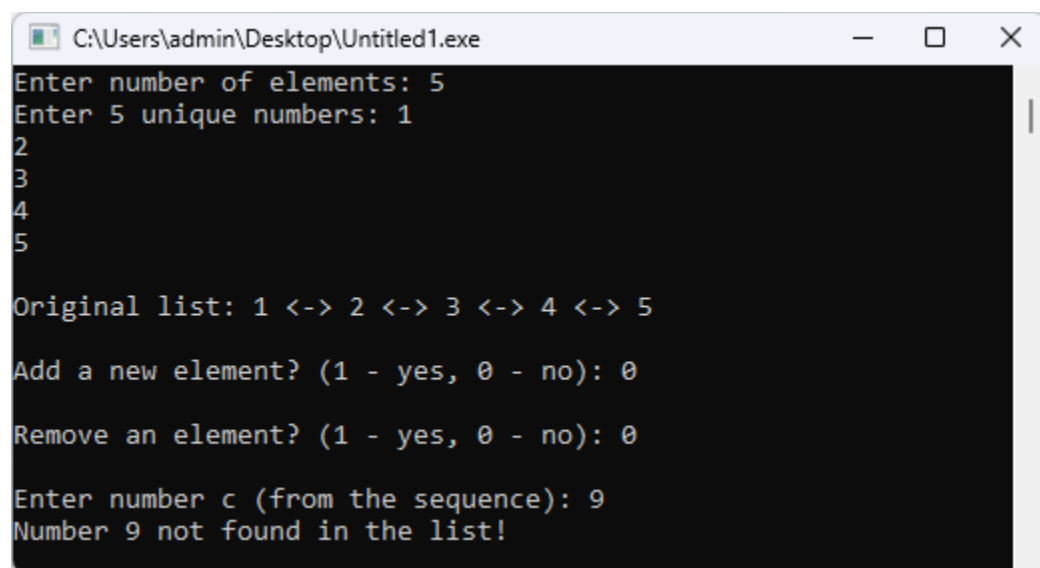
Add a new element? (1 - yes, 0 - no): 1
Enter value: 10
List after adding: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> 7 <-> 10

Remove an element? (1 - yes, 0 - no): 1
Enter value to remove: 3
Element removed.
List after removal: 1 <-> 2 <-> 4 <-> 5 <-> 6 <-> 7 <-> 10

Enter number c (from the sequence): 4

=== Results ===
Numbers before 4 in reverse order: 2 <-> 1
Numbers after 4 in direct order: 5 <-> 6 <-> 7 <-> 10
```

Рисунок 1 – Пример работы программы с добавлением и удалением элементов



```
C:\Users\admin\Desktop\Untitled1.exe
Enter number of elements: 5
Enter 5 unique numbers: 1
2
3
4
5

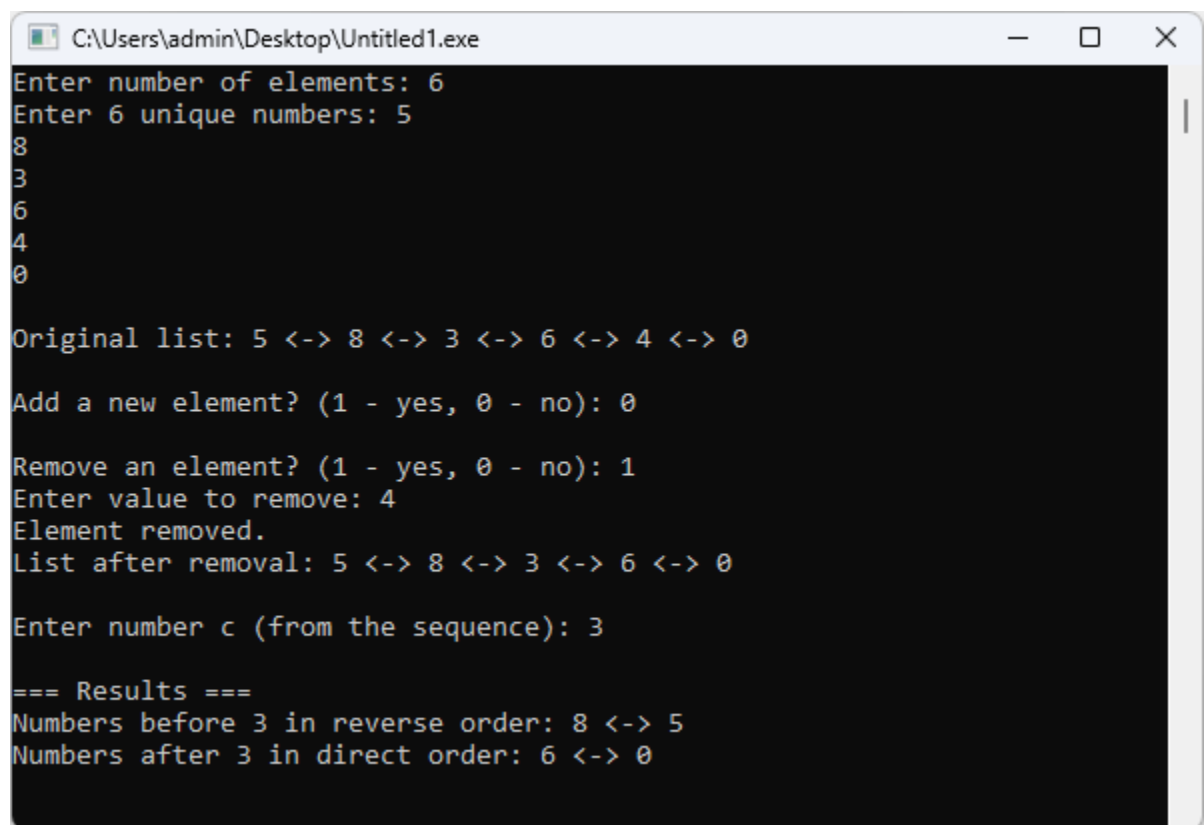
Original list: 1 <-> 2 <-> 3 <-> 4 <-> 5

Add a new element? (1 - yes, 0 - no): 0

Remove an element? (1 - yes, 0 - no): 0

Enter number c (from the sequence): 9
Number 9 not found in the list!
```

Рисунок 2 – Поиск элемента которого нет



```
C:\Users\admin\Desktop\Untitled1.exe
Enter number of elements: 6
Enter 6 unique numbers: 5
8
3
6
4
0

Original list: 5 <-> 8 <-> 3 <-> 6 <-> 4 <-> 0

Add a new element? (1 - yes, 0 - no): 0

Remove an element? (1 - yes, 0 - no): 1
Enter value to remove: 4
Element removed.
List after removal: 5 <-> 8 <-> 3 <-> 6 <-> 0

Enter number c (from the sequence): 3

=== Results ===
Numbers before 3 in reverse order: 8 <-> 5
Numbers after 3 in direct order: 6 <-> 0
```

Рисунок 3 – Удаление элемента

6 Сравнение с кодом на Kotlin

6.1 Листинг программного кода

```
import java.util.Scanner

// Класс узла двусвязного списка
class Node(var data: Int) {
    var prev: Node? = null // ссылка на предыдущий элемент
    var next: Node? = null // ссылка на следующий элемент
}

// Класс для хранения головы и хвоста списка
class DoublyLinkedList {
    var head: Node? = null
    var tail: Node? = null

    // Добавление элемента в конец списка
    fun addToEnd(value: Int) {
        val newNode = Node(value)
        newNode.prev = tail

        // если список не пуст
        if (tail != null) {
            tail!!.next = newNode
        } else {
            head = newNode // если список был пуст
        }

        tail = newNode
    }

    // Удаление элемента по значению
    fun removeByValue(value: Int): Boolean {
        var cur = head

        // поиск элемента
        while (cur != null) {
            if (cur.data == value) {

                // перенастройка ссылок
                if (cur.prev != null)
                    cur.prev!!.next = cur.next
                else
                    head = cur.next

                if (cur.next != null)
                    cur.next!!.prev = cur.prev
            }
            cur = cur.next
        }
    }
}
```

```

        else
            tail = cur.prev

        return true
    }

    cur = cur.next
}

return false
}

// Печать списка
fun printList() {
    if (head == null) {
        println("List is empty")
        return
    }

    var cur = head

    while (cur != null) {
        print(cur.data)

        if (cur.next != null)
            print(" <-> ")

        cur = cur.next
    }

    println()
}

// Очистка списка
fun freeList() {
    head = null
    tail = null
}
}

fun main() {

    val scanner = Scanner(System.`in`)

    val list = DoublyLinkedList()

    // ввод количества элементов
    print("Enter number of elements: ")

```

```

val n = scanner.nextInt()

print("Enter $n unique numbers: ")

for (i in 0 until n) {
    val x = scanner.nextInt()
    list.addToEnd(x)
}

print("\nOriginal list: ")
list.printList()

// добавление элемента
print("\nAdd a new element? (1 - yes, 0 - no): ")
var choice = scanner.nextInt()

if (choice == 1) {
    print("Enter value: ")
    val valAdd = scanner.nextInt()

    list.addToEnd(valAdd)

    print("List after adding: ")
    list.printList()
}

// удаление элемента
print("\nRemove an element? (1 - yes, 0 - no): ")
choice = scanner.nextInt()

if (choice == 1) {

    print("Enter value to remove: ")
    val valRemove = scanner.nextInt()

    if (list.removeByValue(valRemove))
        println("Element removed.")
    else
        println("Element not found.")

    print("List after removal: ")
    list.printList()
}

// ввод числа c
print("\nEnter number c (from the sequence): ")
val c = scanner.nextInt()

```

```

// поиск элемента c
var cNode = list.head

while (cNode != null && cNode.data != c)
    cNode = cNode.next

if (cNode == null) {
    println("Number $c not found in the list!")
    return
}

// --- первая последовательность: элементы до c в обратном порядке --
-
val resList1 = DoublyLinkedList()

var cur = cNode.prev

while (cur != null) {
    resList1.addToEnd(cur.data)
    cur = cur.prev
}

// --- вторая последовательность: элементы после c в прямом порядке -
--
val resList2 = DoublyLinkedList()

cur = cNode.next

while (cur != null) {
    resList2.addToEnd(cur.data)
    cur = cur.next
}

// вывод результатов
println("\n=== Results ===")

print("Numbers before $c in reverse order: ")
resList1.printList()

print("Numbers after $c in direct order: ")
resList2.printList()

// очистка списков
list.freeList()
resList1.freeList()
resList2.freeList()
}

```

6.2 Пример работы программы

```
Enter number of elements: 5
Enter 5 unique numbers: 1
3
5
7
2

Original list: 1 <-> 3 <-> 5 <-> 7 <-> 2

Add a new element? (1 - yes, 0 - no): 1
Enter value: 10
List after adding: 1 <-> 3 <-> 5 <-> 7 <-> 2 <-> 10

Remove an element? (1 - yes, 0 - no): 0

Enter number c (from the sequence): 2

=== Results ===
Numbers before 2 in reverse order: 7 <-> 5 <-> 3 <-> 1
Numbers after 2 in direct order: 10
```

Рисунок 4 – Работы программы на Kotlin

6.3 Сравнение программы на Kotlin и C++

6.3.1 C++

- низкоуровневый контроль
- ручное управление памятью
- больше кода для структур данных

6.3.2 Kotlin

- высокоуровневый
- автоматическое управление памятью
- много готовых коллекций и функций

Итого код на Kotlin получается более простой к пониманию и вниканию, а также один и тот же алгоритм на Kotlin обычно короче на 30–60%, однако ценой таких возможностей часто является высокая абстракция, накладные расходы в виде оперативной памяти и процессорного времени

7 Выводы по работе

В ходе выполнения лабораторной работы была изучена структура данных «линейный двусвязный список» и получены практические навыки её реализации на языке C++.

В отличие от односвязного списка, двусвязный список хранит указатели на оба соседних узла, что даёт возможность перемещаться по списку в обоих направлениях. Это свойство оказалось ключевым при решении поставленной задачи: получение элементов до числа *c* в обратном порядке было реализовано простым обходом по указателям *prev* без дополнительных вспомогательных структур и лишних итераций.

Были реализованы операции добавления элемента в конец списка, удаления элемента по значению и вывода списка на экран. Программа корректно обрабатывает граничные случаи: отсутствие элементов до или после числа *c*, а также ситуацию, когда указанное число не найдено в последовательности.

Также проведено сравнение с аналогичной реализацией на Kotlin